



南京大學

本科畢業論文

学 院	智能软件与工程学院
专 业	软件工程（智能化软件）
题 目	实践中的理论最优哈希表
年 级	2022 学 号 221900323
学生姓名	纪泽操
指导教师	刘明谋 职 称 副教授
提交日期	2026 年 5 月 20 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：实践中的理论最优哈希表

学院：智能软件与工程学院

专业：软件工程（智能化软件）

本科生姓名：纪泽操

指导教师（姓名、职称）：刘明谋 副教授

摘要：

开放寻址哈希表可归结为在线“球与槽”分配：每个键沿固定随机探针序列寻找槽位，插入与删除可移动已有键；平均探针复杂度与交换成本构成一对核心折中。本文依“底层驱逐上界—中间层紧凑编码—顶层组装”的顺序整理 OTSH 型构造在工程中的展开方式：以 k-kick 树在分层 bin 上做贪心优先级驱逐，使单次插入交换次数不超过 $O(k)$ ，并在随机模型下将平均探针控制在 $O(\log^{(k)} n)$ 量级；继而给出 MiniArray 与局域查询路由器，再说明以 Cubby 封装 polylog 规模的存储数组、变长元数据与空闲槽位树，以 Facility 聚合约 $K = \text{polylog } N$ 个键的路由 MiniArray D 与多 tier 小隔间，并用唯一 tail 维持“除尾外全满”的饱和度不变量；以分布目标 t_j 触发创建/销毁重建与双表扩容缩容；以商压缩隐式携带路由高位，使载荷空间接近信息论下界。

实现上采用 C++20 单进程 HTTP 服务 (cpp-httpplib), MySQL 持久化键集与 otsh_op_metrics 操作指标，并附带无数据库依赖的 otsh_perf 基准与 otsh_tests 回归；热路径对指标采用内存缓冲、达千条或显式 flush 时批量写入，结构快照由 /api/analytics/structure_dump 等接口触发。本文为当前仓库中与理论构件对齐的模块划分、表结构与运行配置提供可直接对照的叙述。

关键词：哈希表；k-kick 树；商压缩；多 tier 重建；MySQL 指标

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: On the Optimal Time/Space Tradeoff for Hash Tables

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software Engineering (Intelligent Software)

UNDERGRADUATE: Ji Zecao

MENTOR: Liu Mingmou, Associate Professor

ABSTRACT:

Open-addressing hash tables can be viewed as an online balls-into-bins process: each key follows a fixed random probe sequence, and inserts/deletes may relocate existing keys, trading average probe complexity against swap cost. Following the same bottom-up organization (eviction bounds, compact metadata, global assembly), this thesis presents an engineering-oriented account of an OTSH-style construction: a k-kick tree performs greedy prioritized eviction over layered bins, bounding swaps per insert by $O(k)$ while targeting $O(\log^{(k)} n)$ average probes in the random model; a Cubby packs a polylog-sized slot array, variable-length MiniArray metadata, and a free-slot tree; a Facility aggregates roughly $K = \text{polylog } N$ keys via a routing MiniArray D and multi-tier cubbies with a unique tail enforcing an almost-everywhere-full saturation invariant; tier-count targets t_j trigger split/merge rebuilds alongside doubled/halved resizing; quotienting hides routing bits to approach information-theoretic space.

The implementation ships a C++20 single-process HTTP server (cpp-http-lib), MySQL for the key table and an `otsh_op_metrics` fact table, plus a MySQL-free `otsh_perf` benchmark and `otsh_tests`; hot paths buffer per-operation metrics and bulk-insert on threshold or explicit flush, while structural dumps are triggered via `/api/analytics/structure_dump`. The thesis records terminology and schema aligned with this repository for reproducible experimentation.

KEYWORDS: Hash table; k-kick tree; Quotienting; Multi-tier rebuild; MySQL metrics

目 录

目 录	III
第一章 引言	1
1.1 研究背景	1
1.2 本文目标与范围	1
1.3 主要贡献	2
1.4 论文结构	2
第二章 问题形式化与 k-kick 树上界	3
2.1 球与槽的在线分配	3
2.2 开放寻址与探针复杂度	3
2.3 k-kick 树：分层 bin 与优先级驱逐	3
2.4 本章小结	4
第三章 紧凑构件与系统总体设计	5
3.1 设计目标与复杂度轮廓	5
3.2 紧凑元数据与局域查询路由	5
3.2.1 MiniArray：变长条目的紧凑动态数组	5
3.2.2 Local Query Router：少量键到槽索引的映射	6
3.3 三层架构：从 Cubby 到 Facility	6
3.3.1 Cubby（微观层）	6
3.3.2 Facility 与路由层	6
3.3.3 全局状态与动态容量	7
3.4 分布不变量与重建	7
3.5 插入、查询与删除主路径	7

3.6	商压缩与键重建	8
3.7	模块依赖关系	8
第四章	实现要点	9
4.1	技术栈与交付物	9
4.2	运行形态与并发模型	9
4.3	配置与存储入口	9
4.4	核心数据结构映射	10
4.5	MiniArray 与路由器（工程形态）	10
4.6	操作主路径与指标	11
4.7	HTTP 与分析接口（摘要）	11
第五章	实验与观测设计	12
5.1	实验目的	12
5.2	快照与维度表	12
5.3	槽位与 tier 聚合	12
5.4	指标表与典型 SQL	13
5.5	写入策略	13
5.6	无数据库基准	13
5.7	自动化测试与日志	13
5.8	可视化	14
5.9	本章小结	14
第六章	总结与展望	15
6.1	工作总结	15
6.2	局限与后续工作	15
6.3	结束语	15
	参考文献	16
	致 谢	17
	附录 A 接口与数据模式补充	18

A.1	主要 HTTP API (与 <code>http_server.cpp</code> 一致)	18
A.2	关系模式 (由 <code>ensure_analytics_ddl</code> 创建)	19
A.2.1	<code>snapshot_meta</code>	19
A.2.2	<code>facility</code> 、 <code>cubby</code> 、 <code>slot_snapshot</code> 、 <code>tier_stat</code>	19
A.2.3	<code>otsh_op_metrics</code> (操作级事实表)	21
A.2.4	<code>otsh_test_session_log</code>	21
A.3	清空语义	22
A.4	运行配置 (环境变量)	22

第一章 引言

1.1 研究背景

开放寻址哈希表可抽象为在线的“球与槽”分配问题：给定 n 个槽与动态到达的球（键），每个球附带一条固定的随机探针序列，只能落在序列中的某一槽；若球最终落在其探针序列的第 i 项，则其探针复杂度可理解为 $\Theta(1 + \log i)$ 量级，对应为记录“使用了序列中第几个位置”所需的额外比特。与此同时，插入与删除可能移动已有球，其移动总量构成交换成本。如何在平均探针复杂度与交换成本之间取得折中，是高性能键值映射结构的根本问题之一。

Bender 等在 arXiv:2111.00602（后文记为 OTSH 文献）中给出了哈希表时间—空间折中的完整曲线：对任意 $k \in [\log^* n]$ ，可同时实现 $O(k)$ 时间的插入/删除、 $O(1)$ 时间的查询，以及 $O(\log^{(k)} n)$ 量级的每键浪费比特（均高概率于 n ）^[1]。构造通过分层 bin、随机优先级与级联驱逐（k-kick 树）组织槽位占用，并配合紧凑元数据编码与商压缩，使总空间接近信息论意义下的最优量级。将上述理论构件落实为可运行、可观测、可实验的工程系统，对理解其实际行为与验证分布性质具有明确价值。

1.2 本文目标与范围

本文围绕 OTSH 文献中的理论目标，系统阐述从“探针复杂度—交换成本”模型到完整哈希表的三层架构：微观层（Cubby 内的 k-kick 树与元数据）、路由层（设施内的 MiniArray D 与 Local Query Router）、宏观层（多 tier 小隔间、tail 不变量、后台重建与扩容缩容）。在此基础上，说明本课题**当前仓库**的工程形态：以 C++20 单进程 HTTP 服务（cpp-httpplib）暴露实验接口；以 MySQL 持久化键集与 otsh_op_metrics 等指标表，并在服务端自动建表；另提供无 MySQL 依赖的 otsh_perf 与 otsh_tests 便于本地验收。可视化可由任意 HTTP 客户端或自研前端完成，本文不绑定具体 UI 框架。

本文不重新证明原文定理，而侧重结构分解、不变量、操作流程与实现映射，使读者能够依据本文复现或扩展同一类系统。

1.3 主要贡献

- 将球与槽模型、k-kick 树分层结构、级联驱逐与 $O(k)$ 交换上界整理为第二章，并与 arXiv:2111.00602 的折中曲线表述对齐；
- 将 MiniArray、Local Query Router 独立为第三章的前置节，再给出 Cubby、Facility、全局状态与双表迁移/重建调度的职责划分及插入/查询/删除主路径；
- 阐述分布不变量 t_j 、创建/销毁重建及其均摊意义，以及商压缩下键的重建思路；
- 归纳面向实验验证的 MySQL 关系模式(快照、cubby、tier 统计、otsh_op_metrics)与内存缓冲批量落库策略。

1.4 论文结构

全文依“底层驱逐上界—中间层紧凑编码—顶层组装”展开：第二章从球与槽模型出发，形式化探针与交换成本，并归纳 k-kick 树分层驱逐与文献给出的折中上界；第三章先给出 MiniArray 与 Local Query Router 两类紧凑构件，再给出 Cubby、Facility 与全局扩容/重建流程及商压缩键重建；第四章对应 C++/HTTP 实现与代码映射；第五章给出 MySQL 快照与 otsh_op_metrics 观测设计及典型分析查询；第六章总结并讨论局限与展望。附录给出 HTTP 接口与关系模式等补充材料。

第二章 问题形式化与 k-kick 树上界

2.1 球与槽的在线分配

设共有 n 个槽，任意时刻占用球数不超过 n 。每个球 x 在插入前即固定一条随机探针序列，球只能被安置在该序列中的某一槽；允许在插入或删除时移动其他球。评价指标包括：

- 当前所有球的探针复杂度的平均值（与额外空间、查询路径相关）：若球 x 最终落在其探针序列的第 i 项，则探针复杂度可理解为 $\Theta(1 + \log i)$ 量级，对应为记录“使用了序列中第几个位置”所需的约 $\log i$ 比特；
- 单次插入或删除过程中被移动的球的个数之和（交换成本，与操作时间相关）。

最优折中构造即是在两类指标间给出可证明的上界与（在一定结构类上的）匹配意义下的紧界。

2.2 开放寻址与探针复杂度

在开放寻址哈希表中，键 x 的查找沿探针序列 $h_1(x), h_2(x), \dots$ 依次检查槽位，直至命中或判定不存在。上一节的“球与槽”模型即其动态版本：插入与删除可重排已有键，使“落槽位置”不再限于首次空位。当表在高负载因子下运行时，冲突加剧，探针序列变长；若同时要求插入/删除代价可控，则往往需要在“移动已有元素”（交换）与“压低探针指标”之间权衡。

2.3 k-kick 树：分层 bin 与优先级驱逐

在 Bender 等人的 OTSH 构造中^[1]， n 个槽被组织为深度 $k + 1$ 的分层 bin 树：第 0 层为全长数组，更深层 bin 尺寸按迭代对数迅速减小（本实现取各层尺寸为

$\Theta((\log^{(i)} n)^6)$ 量级，与文献分层尺度同阶)。每个球在插入前随机确定：其各层祖先 $\text{bin } g_0(x), \dots, g_k(x)$ ；最高允许深度 $s(x) \in [0, k]$ ，且 $\Pr[s(x) < i] = 1/(\log^{(i)} n)^2$ ，用于定义优先级；以及沿 bin 从深到浅拼接得到的探针遍历顺序。

插入时自最深允许层向下扫描，寻找第一个“未饱和”的祖先 bin ：饱和指 bin 已满且其中所有球的优先级均不低于该层。若 bin 未满则直接放入；若满但未饱和，则其中必存在来自更浅层的低优先级球，随机踢出一枚并由被踢球在其所属层继续同样规则，形成级联驱逐。每层至多踢出一次，故单次插入交换次数 $\leq k + 1$ 。在随机模型下，平均探针复杂度可达到 $O(\log^{(k)} n)$ 量级；每将插入/删除时间增加一个加法常数，浪费比特可指数下降，这是原文的核心折中曲线^[1]。本文不重新证明该曲线，而以此为上界目标组织工程结构。

2.4 本章小结

本章从在线“球与槽”出发，固定了探针复杂度与交换成本两个量，并给出 k -kick 树的分层、优先级与级联驱逐机制，以及 $O(k)$ 次交换的上界直觉。第三章将在同一章内先给出 MiniArray 与 Local Query Router 两类紧凑构件，再给出 Cubby、Facility 与全局插入/查询/删除流程。

第三章 紧凑构件与系统总体设计

3.1 设计目标与复杂度轮廓

本系统实现开放寻址型动态哈希表的一层工程化外壳,其算法目标与文献一致:查询时间为确定性 $O(1)$ (在路由器与元数据访问为常数时间的理想模型下);插入与删除为 $O(k)$ 次交换 (高概率意义由原文保证);总空间为 $nw + O(n \log^{(k)} n)$ 比特量级,其中 w 为值宽相关项,第二项来自路由与插入元数据^[1]。工程实现中,常数因子与 I/O、锁竞争、批量落库策略会叠加在实际延迟之上。

3.2 紧凑元数据与局域查询路由

当 bin 与 cubby 尺度为 $\text{polylog } n$ 时,需在总空间接近信息论线性的前提下存储变长元数据并支持常数时间读写;同时需在设施内把键映射到“目标 cubby 与槽位”。以下两个构件构成文献构造中的“中间层”,负责在 $\text{polylog } n$ 尺度内紧凑表示与常数时间访问。

3.2.1 MiniArray: 变长条目的紧凑动态数组

每个 cubby 内需维护长度约为 $\text{polylog } n$ 的条目数组,但每条元数据的比特长度不同且随更新变化。文献与理想化工程叙述中,MiniArray 可视为字对齐的 packed B-树:叶存实际条目比特,内部节点存子块偏移与枢轴 (索引范围);因总条目数为 $\text{polylog } n$,指针与枢轴可用 $\Theta(\log \log n)$ 比特打包进单字,树高 $O(1)$,并支持均摊常数的单点更新与查表加速。当前仓库中的可运行原型采用连续比特缓冲 + 偏移表的变长编码实现 (见第四章),在单 cubby 的 polylog 规模下以简化重打包换取实现复杂度;复杂度轮廓与信息论目标仍以本节及文献为参照。

3.2.2 Local Query Router: 少量键到槽索引的映射

在设施的 D 数组及 cubby 内部, 需对规模至多为 $O(\log n / \log \log n)$ 的键集合回答“键 x 落在何处”。实现上对键哈希为足够长的随机二进制串, 构建压缩前缀二进制 Trie: 仅在能区分彼此的最短唯一前缀处停止, 叶部存储 cubby 标识与槽索引等变长元数据; 整棵 Trie 按深度优先遍历编码为紧凑比特流。查询时沿查询键的哈希串在编码流上行走, 配合按半字 (或半字宽) 查表实现期望常数时间的跳转; 插入与删除则更新 Trie 与编码流。对给定元素个数, 编码期望长度为 $O(\log n)$ 比特量级 (高概率不超过同阶上界)。

3.3 三层架构: 从 Cubby 到 Facility

第二章已给出 k-kick 树上界; 第 3.2 节已给出两类紧凑构件。本节给出完整表的逻辑分层, 与实现中的模块边界一致。

3.3.1 Cubby (微观层)

每个 Cubby 是规模约为 $\text{polylog } N$ 的独立 k-kick 实例, 包含:

- 载荷数组 S : 长度 $|I|$, 存放键值载荷或商压缩后的后缀;
- MiniArray Meta: 每槽一条变长 MetaEntry, 聚合查询路由辅助、插入控制 (深度、bin 饱和相关) 与商压缩修复信息;
- FreeSlotTree: 浅层位图树, $O(1)$ 深度查找空闲槽。

tier 与容量由全局 N, K 及迭代对数导出: 容量近似为 $K/(\log^{(\text{tier}+1)} N)^2$ 形式, 用于控制单层 cubby 规模。

3.3.2 Facility 与路由层

全表划分为约 N/K 个设施 (Facility), 每个设施聚合约 $K = \text{polylog } N$ 个键的路由与 cubby 集合。设施内维护:

- D : 长度为 K 的路由器表, 每项为一个 Local Query Router (代码中为 `std::vector<Router` 与“按桶索引的紧凑路由阵列”同构); 对键 x , 由 $\pi(x)$ 的低 $\log K$ 位索引到 $D[b]$, 得到目标 cubby 与槽位;

- 多 tier 的 cubby 列表及 **tail**: 除唯一 tail 外, 其余 cubby 均保持满; 插入一律进入 tail, 删除非 tail 槽时从 tail 随机搬迁一枚元素填补, 以维持饱和度不变量。

设施索引由 $\pi(x)$ 的更高位段与全局 N, K 共同决定 (商压缩模式下 $g^*(x) = \pi(x) \bmod N$ 时, 低 $\log N$ 位拆分出设施与桶索引)。

3.3.3 全局状态与动态容量

全局维护当前键数 n 、容量上界 N (取 2 的幂)、 K 及设施向量; **双表迁移** (活跃 TableState 与可选的旧表) 在 n 超过负载因子上界或低于 $N/4$ 时触发倍增/减半, 并按预算步数渐进搬迁; RebuildScheduler 与 tier 计数驱动的重建/合并逻辑在设施内维护分布不变量 (与第四章实现细节对应)。

3.4 分布不变量与重建

设 n 为当前规模, 第 j 级 tier 的 cubby 目标数量近似为

$$t_j \approx \frac{(\log^{(j)} n)^2}{(\log^{(j+1)} n)^2},$$

第 j 级 cubby 容量尺度约为 $r_j \approx K/(\log^{(j)} n)^2$ (与上文 t_j 配套使用)。当某 tier 计数小于 $t_j/2$ 或大于 $3t_j$ 时触发后台重建: 过少则将一个 $j+1$ 级 cubby 拆为约 t_j 个 j 级 cubby; 过多则将约 t_j 个 j 级 cubby 合并为一个 $j+1$ 级 cubby。单次重建代价 $O(|I|)$, 触发间隔 $\Theta(|I|)$ 次操作, 故均摊附加时间为 $O(1)$ 。

3.5 插入、查询与删除主路径

以下均先应用可逆随机置换 π (工程上可用 Feistel 型 32–32 轮函数实现), 记 $y = \pi(x)$ 。

插入。 由 y 的低 $\log N$ 位划分设施 r 与路由桶 b ; 在 $D[b]$ 中确认不存在后, 将键插入设施 r 的 tail: 在 tail 内执行 k-kick 插入得槽位 slot, 写 $S[\text{slot}]$ 与 Meta, 更新空闲树, 再在路由器中登记指纹到 $(\text{tail}, \text{slot})$ 。若 tail 变满则新建

tail; n 递增, 必要时扩容。

查询。同样计算 (r, b) , 经 $D[b]$ 得到 (I, i) , 读出 $S[i]$ 与 Meta, 用商压缩字段与设施编号重建 g^* 与完整 $\pi(x)$, 再 π^{-1} 校验并返回值。

删除。定位后清空槽与路由器; 若 I 非 tail, 则从 tail 随机取一枚迁入空槽; 若 tail 空则回收并另选一满 1-tier cubby 提升为 tail; n 递减, 必要时缩容。

3.6 商压缩与键重建

令 $g^*(x) = \pi(x) \bmod N$, 则顶 $\log N$ 位可由路由过程隐式携带。Cubby 内 S 不重复存这 $\log N$ 位, 而在 Meta 中记录例如 $\delta = g_I(x) - i$ (偏好槽与实际槽之差) 及 mid_bits (衔接 g_I 与 g^* 低段所需的中间位)。查询时由 r 、 g_I 与 mid_bits 拼出 g^* , 与 S 中高位载荷合并得 $\pi(x)$, 再逆置换得原键。

3.7 模块依赖关系

逻辑依赖可概括为: $\text{HashTable} \rightarrow \text{Facilities} \rightarrow (D, \text{Cubbies}, \text{tail})$, Cubby 内部依赖 $(S, \text{Meta}, \text{FreeSlotTree})$ 与 k-kick 插入; 全局依赖双表迁移状态机与 Re-buildScheduler。该划分与 ht.cpp、otsh/facility.h、otsh/cubby.h 等源文件边界相对应。

第四章 实现要点

4.1 技术栈与交付物

本仓库以 CMake(要求 ≥ 3.20) 组织构建:核心为 C++20 可执行文件 `otsh_backend` (HTTP 服务)、`otsh_tests` (内存表回归并可写 MySQL 测试日志)、`otsh_perf` (纯内存插入/查询/删除延迟与进程内 Metrics 汇总,不链接 MySQL)。HTTP 层依赖单头库 `cpp-httpplib`^[2] 与 `nlohmann/json`^[3];可选置于 `third_party` 或通过 `FetchContent` 获取。持久化与观测走 MySQL C API (`libmysqlclient`/MariaDB 兼容),由 `src/storage_mysql.cpp` 在首次连接时 `CREATE TABLE IF NOT EXISTS` 分析侧各表。

4.2 运行形态与并发模型

`otsh_backend` 内持有一个全局 `std::mutex (service_mu)`,对 `/api/init` 与所有写路径 (插入/删除及依赖表状态的实验任务) 加锁,保证与当前“单 writer、可重复实验”目标一致;查询与统计在实现中同样经该锁与表状态交互,行为以代码为准。探活为 `GET /health`,返回纯文本 `ok`。进程日志默认写 `backend.log`,可通过环境变量 `LOG_FILE` 覆盖。监听地址与端口由 `BIND_HOST` (默认 `0.0.0.0`)、`PORT` (默认 `8080`) 指定。

4.3 配置与存储入口

`TableParams (include/config.h)` 描述初始化规模 `n`、占位参数 `k`、`load_factor`、三枚种子 `seed1..seed3` 以及 MySQL 主机/端口/用户/密码/库名/主键表名 (默认表名 `otsh_keys`,单列 `k BIGINT UNSIGNED PRIMARY KEY`)。当前 `/api/init` 请求体主要解析 `n`、`k`、`reset`、`snapshot_tag`,其余字段沿用上述默认值;自动化测试程序另支持 `OTSH_MYSQL_*`

环境变量覆盖连接信息。初始化成功后创建分析用 `snapshot_id` 并作为后续 `otsh_op_metrics` 行的外键上下文。

4.4 核心数据结构映射

对外类型为 `HashTable` (`include/ht.h`, 实现于 `src/ht.cpp` 的 `HashTable::Impl`) 内部以嵌套结构 `TableState` 保存当前表的 N (不小于 2 的 2 的幂)、 K 、`std::vector<Facility>`; 另以 `std::optional<TableState> old_` 与 `migrate_progress_` 实现**双表扩容/缩容迁移**: 当负载超过 `load_factor` 或键数低于 $N/4$ 时分配新表并逐设施预算化搬迁, 迁完后释放旧表。文献中的 `ResizeManager` 在仓库 `include/otsh/resize.h` 中仍为占位类型, 实际迁移逻辑内联于 `HashTable::Impl`。

`RebuildScheduler` (`include/otsh/rebuild.h`) 维护 `std::deque` 任务队列, 在每次插入后的预算步数内执行 `tier` 合并/拆分等后台工作, 对应设计章分布不变量触发逻辑的工程化版本。

`PermutationHash` 将 64 位键高/低各 32 位视作 Feistel 两半, 使用三轮轮函数 (轮密钥取自 `k1..k3`, 由 `splitmix64` 与种子派生); `k4` 与额外种子字段一并派生但**不参与**当前三轮 Feistel 轮函数, 保留作扩展。逆置换 `inverse` 对称实现, 用于校验与调试路径。

每个 `Facility` (`include/otsh/facility.h`) 包含: 按 `tier` 分组的二维 `std::vector`, 内层元素为 `std::unique_ptr<Cubby>`; `tail` 指针与独占 `tail_owned`; 长度为 K 的 `std::vector<Router>` 即设施内路由器数组 `D`。每个 `Cubby` (`include/otsh/cubby.h`) 含 `tier`、`capacity`、`size`、商压缩载荷槽向量 `slots` (元素为 `std::optional<uint64_t>`)、占用索引表、`FreeSlotTree` `free_slots` 与 `MiniArray` `meta`。

4.5 MiniArray 与路由器 (工程形态)

`MiniArray` (`include/otsh/mini_array.h`) 当前实现为: 每条目变长比特写入连续 `buf_`, 以 `offsets_` 与 `bitlens_` 定位; 更新允许对单 `cubby` 规模 m 做 $O(m)$ 重打包, 满足“polylog 规模下可接受”的原型目标, 与设

计章所述 packed B-树理想模型相比更偏工程折中。

Router (include/otsh/router.h, 对应设计章 Local Query Router) 用 64 位哈希串上的二叉 trie 加碰撞桶管理 $\text{key} \rightarrow (\text{Cubby}^*, \text{slot})$; 维护编码比特流规模 `bits_total` 供空间统计。插入与删除后允许整表重建 trie/编码 (元素数为 polylog 时开销可控), 与“均摊常数跳转”的理论目标之间留有优化空间。

4.6 操作主路径与指标

插入/查询/删除主流程与第三章叙述一致: 经 $\pi(\text{key})$ 与设施、桶索引定位 $D[b]$, 在 `tail cubby` 上执行 `k-kick` 相关逻辑并维护路由器与商压缩元数据。每次操作可将 `probe_count/kick_count/latency_ns/cubby_tier` 等封装为 `SqlMetricRow`, 经 `IStorage::analytics_enqueue_metric` 进入缓冲; 缓冲长度达到 1000 或显式 `analytics_flush_metrics(true)` 时, 以最多 500 行一批多值 `INSERT` 写入 `otsh_op_metrics`。进程内另有一套原子计数器 `Metrics (include/metrics.h)` 汇总吞吐、移动步数、延迟直方图与重建/扩容事件, `otsh_perf` 直接读取其快照。

4.7 HTTP 与分析接口 (摘要)

除 `/api/init` 外, 实现还提供 `/api/insert`、`/api/query`、`/api/delete`、`/api/stats`; 分析侧含 `/api/analytics/snapshots`、`/api/analytics/summary`、`/api/analytics/structure_dump`; 长任务轮询 `/api/jobs/get`; 参数扫描实验 `/api/experiment/ol_vs_n`、`/api/experiment/ok_vs_k`。详细路由与 JSON 字段见附录 A。

第五章 实验与观测设计

5.1 实验目的

在理论结构固定而工程参数可调的前提下，实验关注三类可观测量：（1）探针与踢出次数沿操作类型的分布；（2）各 tier cubby 的负载因子与 cubby 数量是否贴近第三章给出的 t_j 目标；（3）tail 行为与延迟尾部是否与理论假设一致。数据库侧以 otsh_op_metrics 为核心事实表，快照维表记录某次 snapshot_id 下的结构导出结果，便于脚本或外部可视化工具复现图表。

5.2 快照与维度表

snapshot_meta 标记一次 /api/init 或手工标记的快照：字段 init_param_n、table_n、table_k 分别对应初始化请求规模与当时内存表 N, K （命名规避 Windows 下大小写不敏感导致的列名冲突）。facility 记录设施 id、所属快照与 tail_cubby_id。cubby 记录 tier、capacity、size、is_tail 等，用于展示 tier 分布与利用率。

5.3 槽位与 tier 聚合

slot_snapshot 按快照存储 cubby_id、slot_index、是否占用、可选 key_hash（实现中取 $\pi(x)$ ）及 probe_length。该表刻意不存比特级 Meta 与路由器编码，以降低快照体积并聚焦“最终落槽”结果。tier_stat 按 (snapshot_id, facility) 聚合 cubby_count，用于对照

$$t_j \approx \frac{(\log^{(j)} n)^2}{(\log^{(j+1)} n)^2}.$$

5.4 指标表与典型 SQL

注意：事实表名为 `otsh_op_metrics` (非 `metrics`)。字段包括 `operation_type` (`insert/query/delete`)、`probe_count`、`kick_count`、`latency_ns`、`cubby_tier`、`snapshot_id`。服务端聚合查询实现见 `analytics_summary_json`，典型分析包括：`SELECT AVG(probe_count) FROM otsh_op_metrics WHERE snapshot_id=?`；按 `cubby_tier` 分组平均探针；在 `cubby` 表上按 `tier` 求 `AVG(size/capacity)`；探针与踢出次数直方图等。

5.5 写入策略

高频路径禁止“每操作一次即单独 `INSERT`”。实现将 `SqlMetricRow` 先入内存 `vector`，当积压达到 1000 条或显式 `flush` 时，以最多 500 行一批拼接多值 `INSERT` 写入 `otsh_op_metrics`。结构快照由 `/api/analytics/structure_dump` 触发：遍历 `HashTable::visit_structure`，再事务性替换对应 `snapshot_id` 下的 `slot_snapshot/cubby/facility/tier_stat` 行。

5.6 无数据库基准

`otsh_perf` 可脱离 `MySQL` 直接测量纯内存插入/查询/删除延迟，并输出进程内 `Metrics` 快照（人类可读信息走 `stderr`，单行 `JSON` 走 `stdout`，便于 `perf/results/` 目录收集）。适合作为算法热路径回归，与带库延迟的 `HTTP` 实验互为补充。

5.7 自动化测试与日志

`otsh_tests` 对哈希表执行初始化与批量操作检查，结束时打印一行 `JSON`；若 `MySQL` 可用则写入 `otsh_test_session_log`，否则可写文本日志（环境变量 `OTSH_TEST_LOG`）。

5.8 可视化

外部工具可读取 `/api/analytics/snapshots` 与 `/api/analytics/summary` 返回的 JSON，绘制平均探针、`tier` 对比、直方图与尾延迟曲线（可选用 ECharts^[4] 等组件库）。图表与第三章中的不变量叙述一并阅读，可判断实现是否偏离理论分布。

5.9 本章小结

实验设计围绕“可证明性质的可观测代理量”展开：用 `otsh_op_metrics` 刻画操作微观代价，用 `tier_stat` 与 `cubby` 刻画宏观分布，用 `/api/analytics/structure` 保证结构可重复导出。具体数值结果随运行环境与数据集变化，本文不绑定某一组绝对数字。

第六章 总结与展望

6.1 工作总结

本文对 OTSH 型最优时间/空间折中哈希表进行了自洽的论文化整理：第二章从球与槽模型与探针复杂度出发，说明 k-kick 树如何通过分层 bin、随机优先级与级联驱逐在 $O(k)$ 次交换内实现文献意义下的折中目标；第三章将 MiniArray 与 Local Query Router 作为独立中间层叙述后，再给出 Cubby、Facility、全局置换与商压缩下的插入/查询/删除主路径及分布不变量。实现层面概述了 C++20/HTTP、MySQL 键表与 otsh_op_metrics、自动 DDL、otsh_perf/otsh_tests 及指标批量写入策略，使结构行为可被记录并与外部可视化解耦。

6.2 局限与后续工作

- 理论命题的严格概率陈述与证明仍以原文为准^[1]，本文未给出形式化引理证明链；
- 工程常数（ K 、tier 边界、缓冲批次、迁移调度）对尾延迟与空间占用的影响需系统化基准测试；
- MiniArray 与 Trie 路由器的查表宽度、缓存友好性与并行安全可进一步专项优化；
- 可探索将快照与指标导出与离线仿真器对接，以分离算法与 I/O 因素。

6.3 结束语

将分层驱逐、紧凑编码与多 tier 重建统一在同一实现中，有助于在数据库与服务化场景下验证“理论最优结构”的实际轮廓。希望本文为后续开发与论文撰写提供一致术语与结构索引。

参考文献

- [1] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. On the Optimal Time/Space Tradeoff for Hash Tables[EB/OL]. (2021-11-04) [2026-05-02]. <https://arxiv.org/abs/2111.00602>. arXiv: 2111.00602.
- [2] HIRAOKA Y. cpp-httpplib[EB/OL]. 2026 [2026-04-27]. <https://github.com/yhironose/cpp-httpplib>.
- [3] LOHMANN N. JSON for Modern C++[EB/OL]. 2026 [2026-04-27]. <https://github.com/nlohmann/json>.
- [4] Apache Software Foundation. Apache ECharts Documentation[EB/OL]. 2026 [2026-04-27]. <https://echarts.apache.org/>.

致 谢

感谢 LUG@NJU。

附录 A 接口与数据模式补充

A.1 主要 HTTP API (与 http_server.cpp 一致)

otsh_backend 通过 HTTP 暴露操作与实验接口；写路径与 /api/init 在实现中与同一互斥锁序列化。路由如下（方法均为所列动词）：

- GET /health: 探活，响应体为纯文本 ok；
- POST /api/init: JSON 体解析 n、k、可选 reset（默认真）、可选 snapshot_tag（默认 init）；校验 $n \in [10, 5\,000\,000]$, $k \in [0, 8]$ 。

打开 MySQL、调用 HashTable::init, 按 reset 清空主键表或 bulk_load 已有键；创建 snapshot_meta 行并刷新指标缓冲。MySQL 连接参数当前取自 include/config.h 中 TableParams 默认值（未从环境变量读取）；

- POST /api/insert: 体为 {"key": <uint64>}（字段名 key）；
- POST /api/query、POST /api/delete: 同上；
- POST /api/stats: 返回内存表状态与进程内 Metrics 汇总；
- POST /api/analytics/snapshots: 列出最近快照元数据（JSON）；
- POST /api/analytics/summary: 体含 snapshot_id, 返回该快照下 otsh_op_metrics 与 cubby 等聚合 JSON；
- POST /api/analytics/structure_dump: 将当前 HashTable::visit_struct 结果写入给定 snapshot_id 关联的 facility、cubby、slot_snapshot、tier_stat 等表；
- POST /api/jobs/get: 轮询异步实验任务状态；
- POST /api/experiment/ol_vs_n、POST /api/experiment/ok_vs_k: 参数扫描实验入口，持续向 otsh_op_metrics 写入样本。

具体错误码与扩展字段以源码为准；可用任意 HTTP 客户端或脚本驱动，无需绑定特定前端框架。

A.2 关系模式（由 ensure_analytics_ddl 创建）

下列表在首次 storage->open 成功时按需创建。主键表名由 TableParams::mysql_t 指定（默认 otsh_keys），单列 k BIGINT UNSIGNED PRIMARY KEY，用于持久化当前键集（与内存结构同步策略以 /api/init 的 reset 与插入/删除路径为准）。

A.2.1 snapshot_meta

为避免 Windows 下 lower_case_table_names 导致列名冲突，规模字段在库中命名为 init_param_n、table_n、table_k：

```
CREATE TABLE IF NOT EXISTS `snapshot_meta` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT,  
  `snapshot_tag` VARCHAR(64),  
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  `init_param_n` BIGINT NOT NULL,  
  `table_n` BIGINT UNSIGNED NOT NULL,  
  `table_k` BIGINT UNSIGNED NOT NULL,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB;
```

A.2.2 facility、cubby、slot_snapshot、tier_stat

```
CREATE TABLE IF NOT EXISTS `facility` (  
  `id` INT NOT NULL,  
  `snapshot_id` BIGINT NOT NULL,  
  `tail_cubby_id` INT NOT NULL,  
  PRIMARY KEY (`id`, `snapshot_id`),  
  KEY `ix_fac_snap` (`snapshot_id`)  
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS `cubby` (  

```

```

`id` INT NOT NULL,
`snapshot_id` BIGINT NOT NULL,
`facility_id` INT NOT NULL,
`tier` INT NOT NULL,
`capacity` INT NOT NULL,
`size` INT NOT NULL,
`is_tail` TINYINT(1) NOT NULL,
PRIMARY KEY (`id`,`snapshot_id`),
KEY `ix_cub_snap` (`snapshot_id`)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS `slot_snapshot`(
  `id` BIGINT NOT NULL AUTO_INCREMENT,
  `snapshot_id` BIGINT NOT NULL,
  `cubby_id` INT NOT NULL,
  `slot_index` INT NOT NULL,
  `occupied` TINYINT(1) NOT NULL,
  `key_hash` BIGINT UNSIGNED NULL,
  `probe_length` INT NOT NULL,
  PRIMARY KEY (`id`),
  KEY `ix_slot_snap` (`snapshot_id`),
  KEY `ix_slot_cubby` (`snapshot_id`,`cubby_id`)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS `tier_stat`(
  `snapshot_id` BIGINT NOT NULL,
  `facility_id` INT NOT NULL,
  `tier` INT NOT NULL,
  `cubby_count` INT NOT NULL,
  PRIMARY KEY (`snapshot_id`,`facility_id`,`tier`)

```

```
) ENGINE=InnoDB;
```

A.2.3 otsh_op_metrics (操作级事实表)

```
CREATE TABLE IF NOT EXISTS `otsh_op_metrics`(  
  `id` BIGINT NOT NULL AUTO_INCREMENT,  
  `snapshot_id` BIGINT NOT NULL,  
  `operation_type` ENUM('insert','query','delete') NOT NULL,  
  `probe_count` INT NOT NULL,  
  `kick_count` INT NOT NULL,  
  `latency_ns` BIGINT NOT NULL,  
  `cubby_tier` INT NOT NULL,  
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (`id`),  
  KEY `ix_met_snap` (`snapshot_id`)  
) ENGINE=InnoDB;
```

实现中缓冲满 1000 条或强制 **flush** 时批量插入，单条 INSERT 语句最多拼接 500 行值。

A.2.4 otsh_test_session_log

供 otsh_tests 在跑完回归后追加一行 JSON:

```
CREATE TABLE IF NOT EXISTS `otsh_test_session_log`(  
  `id` BIGINT NOT NULL AUTO_INCREMENT,  
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  `body` MEDIUMTEXT NOT NULL,  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB;
```

A.3 清空语义

`/api/init` 在 `reset=true` 时调用 `DELETE` 清空上述分析表及主键表（对不存在的历史表名忽略错误），以便重复实验。

A.4 运行配置（环境变量）

- 服务: `BIND_HOST`、`PORT`、`LOG_FILE`;
- 自动化测试 `otsh_tests`: `OTSH_MYSQL_HOST`、`OTSH_MYSQL_PORT`、`OTSH_MYSQL_USER`、`OTSH_MYSQL_PASSWORD`、`OTSH_MYSQL_DATABASE`、`OTSH_MYSQL_TABLE`、`OTSH_TEST_LOG`;
- 基准 `otsh_perf`: `OTSH_SEED` 等（见 `perf/bench.cpp` 注释）。